

Doc link:-

<https://docs.google.com/document/d/1xqNwFtALOCplvjS7mF7yf0pwJXZ9EFJYwiDskFVUdwo/e/dit?usp=sharing>

chapter link:- <https://jax-ml.github.io/scaling-book/inference>

Here are some specific numbers for our chips:

Model	Pod size	Host size	HBM capacity/chip	HBM BW/chip (bytes/s)	FLOPs/s/chip (bf16)	FLOPs/s/chip (int8)
TPU v3	32x32	4x2	32GB	9.0e11	1.4e14	1.4e14
TPU v4p	16x16x1 6	2x2x 1	32GB	1.2e12	2.75e14	2.75e14
TPU v5p	16x20x2 8	2x2x 1	96GB	2.8e12	4.59e14	9.18e14
TPU v5e	16x16	4x2	16GB	8.1e11	1.97e14	3.94e14
TPU v6e	16x16	4x2	32GB	1.6e12	9.20e14	1.84e15

Host size refers to the topology of TPUs connected to a single host (e.g. TPU v5e has a single CPU host connected to 8 TPUs in a 4x2 topology). Here are interconnect figures:

Model	ICI BW/link (one-way, bytes/s)	ICI BW/link (bidi, bytes/s)
TPU v3	1.0e11	2.0e11
TPU v4p	4.5e10	9.0e10
TPU v5p	9.0e10	1.8e11
TPU v5e	4.5e10	9.0e10
TPU v6e	9.0e10	1.8e11

Pop Quiz: Assume we want to take a generate step with a batch size of 4 tokens from a 30B parameter dense model on TPU v5e 4x4 slice in int8 with bf16 FLOPs, 8192 context and 100 kB / token KV caches. What is a reasonable lower bound on the latency of this operation? What if we wanted to sample a batch of 256 tokens?

My Answer:-

so batch size is small so we will be memory bound in both attention and MLP

so time per step= (batch_size*KV_size+parameters_size)/bandwidth
=30GB/4*4bandwidth=30e9/8.1e11/16=2.3ms around

for batch of 256 tokens MLP dominates a bit
so 1st kv cache time = 256*8192*100e3/16/8.1e11=16ms

for ML Player 256>240 so compute bound and so its time is 2*params*batchsize/flops
=2*30e9*256/(16*1.97e14)=4.8ms

so total time is around 21ms

hyperparam	value
L (num_layers)	64
D (d_model)	4,096
F (ffw_dimension)	16,384
N (num_heads)	32
K (num_kv_heads)	8
H (qkv_dim)	256
V (num_embeddings)	32,128

Question 1: How many parameters does the above model have? How large are its KV caches per token in int8? You can assume we share the input and output projection matrices.

My Answer:-

FFL= L*(3*DF)=12.9B

Vocab=2*VD=0.26B

attention= L(2DNH+2DKH)=5.36B

so total param = 18.5B

for KV num kv bytes per token= (bytes per kv param)*L*2KH=262KB

Question 2: Say we want to serve this model on a TPUv5e 4x4 slice and can fully shard our KV cache over this topology. What's the largest batch size we can fit, assuming we use int8 for everything and want to support 128k sequences? What if we dropped the number of KV heads to 1?

My Answer:-

$KV \text{ memory} + \text{model memory} < \text{total memory}$

$BS(\text{mem per KV}) + 18.5e9 < 16 * \text{TPU per chip HBM memory}$

TPU per chip HBM memory = $16e9$

$B * 33e9 < 16 * 16e9 - 18.5e9$

$B \leq 7$

if KV heads is drom to 1 from 8 this becomes 8 times so 56

Question 3: How long does it take to load all the parameters into the MXU from HBM assuming they're fully sharded on a TPU v5e 4x4 slice? Assume int8 parameters.

This is a good lower bound on the per-step latency.

My Answers:-

per chip parameter count = $18.5e9 / 16 = 1.156e9$

HBM->MXU latency = $8.1e11$

so time = 1.427ms

Question 4: Let's say we want to serve this model on a TPUv5e 4x4 slice using int8 FLOPs and parameters/activations. How would we shard it for both prefill and decode? Hint: maybe answer these questions first:

1. What does ICI look like on a 4x4?
2. What's the roofline bound on tensor parallelism?
3. How can we shard the KV caches?

For this sharding, what is the rough per-step latency for generation?

My answer:-

for prefill:-

we only want to find the KV so best strategy for throughput is any of FSDP or magatron based based on batch size or F

lets check for magatron we need $\text{cores} < MxF/2200$ cores ≤ 14 so just commune bound

so we need to do further sequence parallelism and break sam Sequence in 2 so each sequece will effectively see $8 < 14$ in MLP layer so prefill will have full throughput

For generation:-

for model parrallelism to work $T_{\text{ci}} > T_{\text{hbm}}$

which gives us $B > F / (Y * \text{beta})$

$\text{beta} = W_{\text{hbm}} / W_{\text{ci}} = 9$

so $B > 16384 / (16 * 9) = 114$

so if out batch size is large we

we assume we always have B lower than 114

let say we do not have a large batch size

then Thbm is the major bottle neck and we can shard the model in 16 chips with kv sharded across 16 and model sharded across 16 chips

here Thbm is bottle neck so we will roughly require model param/WHBM as time required for 1 step

so $T_{step} = 18.5e9 / 16 * 8.1e11 = 1.4$ ms per token

Question 5: Let's pretend the above model is actually an MoE. An MoE model is effectively a dense model with E copies of the FFW block. Each token passes through k of the FFW blocks and these k are averaged to produce the output. Let's use $E=16$ and $k=2$ with the above settings.

1. How many total and activated parameters does it have? *Activated means used by any given token.*
2. What batch size is needed to become FLOPs bound on TPU v5e?
3. How large are its KV caches per token?
4. How many FLOPs are involved in a forward pass with T tokens?

My answer:-

- 1) total params = $12.9 * 16 + (18.5 - 12.9) = 212B$
active parameters = $12.9 * 2 + (18.5 - 12.9) = 31.5B$
- 2) default batch size is 240 for HBM to be compute bound but now we need to load E models so E times memory and K times computation so final batch size = $240 * E / k = 1920$ tokens
- 3) KV cache is of the same size because only FFW is copied
- 4) for forward pass
its around $T * 2 * (\text{activated_number of params}) = 63e9 * T$ FLOPs

Question 6: With MoEs, we can do "expert sharding", where we split our experts across one axis of our mesh. In our standard notation, our first FFW weight has shape $[E, D, F]$ and we shard it as $[E_z, D_x, F_y]$ where x is only used during training as our FSDP dimension. Let's say we want to do inference on a TPU v5e:

1. What's the HBM weight loading time for the above model on a TPU v5e 8x16 slice with Y=8, Z=16? How much free HBM is available per TPU?

2. What is the smallest slice we could fit our model on?

My Answer:-

$THBM = E * FFW_params / XYWhbm = 1.61B \text{ parameter per chip} / Whbm = 1.65e9 / 8.1e11 = 2ms$
around

so per tpu 1.65 GB is used considering int model so 14.35GB remaining

smallest slice is around model params/16GB so like $212/16 = 14$ chips so 4X4 chip without considering KV cache

Question 7 [2D model sharding]: Here we'll work through the math of what the [ESTI paper](#) calls 2D weight-stationary sharding. We describe this briefly in Appendix B, but try doing this problem first to see if you can work out the math. The basic idea of 2D weight stationary sharding is to shard our weights along both the

D

D and

F

F axes so that each chunk is roughly square. This reduces the comms load and allows us to scale slightly farther.

Here's the algorithm for 2D weight stationary:

1. $In[B, Dx] = AllGathervz(In[B, Dxyz])$
2. $Tmp[B, Fyz] \{Ux\} = In[B, Dx] *_{D} Win[Dx, Fyz]$
3. $Tmp[B, Fyz] = AllReducev(Tmp[B, Fyz] \{Ux\})$
4. $Out[B, Dx] \{Uyz\} = Tmp[B, Fyz] *_{F} Wout[Fyz, Dx]$
5. $Out[B, Dxyz] = ReduceScattervz(Out[B, Dx] \{Uyz\})$

Your goal is to work out

T_{math} and T_{comms}

for this algorithm and find when it will outperform traditional 3D model sharding?

My Answer:-

$$T_{\text{math}} = 4BDF/NC$$

all gather formula is size of moving matrix post merge/ $M \times W_{\text{ici}}$

$$\text{so tot } T_{\text{comm}} = 2BD/2XW_{\text{ici}} + 4BF/YZW_{\text{ici}} + 2BD/2XW_{\text{ici}}$$

solving it we got $X = \sqrt{ND/2F}$

$$F \sim 4D$$

so X is around $\sqrt{N/8}$

$$\text{calculating } T_{2D} = \sqrt{128/N} BD/W_{\text{ici}}$$

for model across 3D it will be $4BD/3W_{\text{ici}}$

so for $N > 81$ we can say 2D is better

here the eq is $F > \sqrt{8N}C/W_{\text{ici}}$ while normally it was $F > NC/W_{\text{ici}} \times \text{num_axis} = NC/3W_{\text{ici}}$

so it is better when $\sqrt{8N}C/W_{\text{ici}} < NC/3W_{\text{ici}}$ means $N > 72$ this way is better

it spends less time in comms